A Mini-Eval for Point-in-Time Correctness

in Data Science

Isolating temporal leakage as a frontier-model failure mode

Adit Rahman

Mini Research Project

June 2026

Summary. This eval measures one skill data scientists use constantly and models keep getting wrong: reconstructing what was actually known at a past moment from data that has since been revised. Five Harbor tasks, each a scaled-down version of a real production workflow, run against `terminus-2 + gemini-3.5-flash` at 12 trials each, for 60 runs total. The set lands at **pass@1 21.7%** and **pass@3 25.0%**, under the 30% target. The interesting part is *why* it fails: in the trajectories the model states the correct point-in-time rule, sometimes almost verbatim, and then talks itself out of applying it — the same way, on nearly every trial. My read is that models have been trained on plenty of material that *describes* this problem and almost none that *solves* it.

1 Where the wedge came from

I started where the brief told me to start: existing DS benchmarks, looking for things `gemini-3.5-flash` would fail on. Most of what I found was dated — no numbers on the model I was actually testing — but it still pointed me somewhere useful, which was the family of failures around *temporal data leakage*: agents consistently underestimate problems where part of the provided data is information from the future, and either don't notice or don't know how to work around it.

The benchmark that convinced me was DataClawBench, a recent process-oriented benchmark for exploratory analysis over real, noisy data (492 cross-domain tasks, native mess preserved), where 7 of 8 agents scored under 50% pass@1. Its scoring is semantic consistency against an oracle solution — effectively a precision/recall style judgment, my read is via an LLM judge — plus an instruction-efficiency metric that punishes flailing. `gemini-3.5-flash` isn't on that leaderboard, but `gemini-3.1-pro` is, and flash outperforms 3.1-pro on agentic benchmarks like DeepSWE. Not a perfect translation, I know, but enough to trust the direction: this family of task is hard for the class of model I'm testing.

There's also a prior I brought into this, which came from a conversation with a friend back when I was first learning ML: models are bad at exactly the things they haven't been trained on. My intuition for how agents reason is that they match the problem in front of them to problems they've seen, and the seen problem becomes a template. If there's no template — or very few — they fall back to the nearest thing they *do* know how to do, which is usually the easier, wrong implementation. I went in with that as a preconception, and by the end of this project I think the trajectories bore it out (§5).

The wedge. So the slice I picked is deterministic point-in-time reconstruction. The cleanest way to picture it: you run a store, and you have two things — the *current state* of every record (what the warehouse says now) and the *ledger* of every event that ever touched those records. The tasks all hinge on translating between those two: rebuilding what a record looked like, or what was knowable, at some past moment, using only events that had been recorded by then and hadn't been superseded by a later correction. A data scientist does this constantly — it's the substrate under every training table — but almost always as a sub-step of a bigger job, and almost exclusively on production systems. Which is exactly why I'd bet there's very little of it in training data: the systems that solve it are proprietary, and the public material mostly just *talks about* it.

The failure this produces has a name, temporal leakage: information that hadn't yet reached the warehouse leaks into a decision's features or labels, the model scores well offline, and it falls apart in production. It's among the most-cited mistakes in applied ML, and production feature stores (Feast, Tecton, Databricks) exist largely to prevent it (§6). This eval grades the skill those tools automate.

The mechanism under every task is bitemporality (Figure 1). Every fact has two timestamps: when it happened (event time) and when it was recorded in the warehouse (recorded time). A feature may use a fact only when *both* fall before the decision. Filtering on event time alone — the obvious shortcut — admits facts that happened early but were recorded late. That's the leak.

Every task is real industry work. Each of the five reproduces a specific job a data scientist does, in a different industry: feature-store training tables, supervised label construction, event-sourced state reconstruction, and performance reporting. The inputs are messy the way real warehouse data is messy — duplicate rows, inconsistent casing, several timestamp formats, records that arrive late. Table 1 lists them.

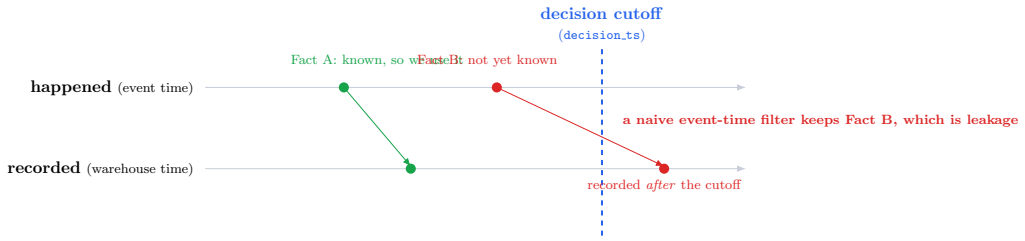


Figure 1: The idea behind every task. Fact B happened before the decision but was recorded afterwards, so it was not knowable at decision time. A correct as-of join filters on both timelines; the naive single-timeline filter lets Fact B leak in.

Task	Domain	What the agent builds
credit-feature-pipeline	Consumer credit	The offline feature table for a credit-risk model: one row per past lending decision, each feature describing the customer as known at that moment.
claims-adjudication-cost-share	Health insurance	The member's cost share (deductible and coinsurance) for each past claim, as known when the claim was adjudicated, including a year-to-date deductible total.
subscription-churn-labels	B2B SaaS	Supervised churn labels: for each observation, did the account churn within 90 days? Observations whose window has not fully elapsed get no label.
order-ledger-reconstruction	E-commerce	The state of each order at past checkpoints, rebuilt from an append-only event log in which later "correction" events can void earlier ones.
portfolio-twr	Finance reporting	The cumulative time-weighted return per account at each month-end. The data is clean; this is the deliberately easy task.

Table 1: The five tasks. Each is a scaled-down version of a real industry workflow.

Scope. In scope: bitemporal as-of joins, slowly-changing-dimension lookups, event-log reconstruction with corrections, right-censored label construction, and windowed aggregates over deduplicated, late-arriving data. Out of scope: model training, plotting, open-ended exploration, anything gradeable only by taste. I stayed where a correct answer exists and can be checked. The rest of the report answers the brief's five questions, mapped here.

Brief question	Short answer	Section
1. Distribution	Point-in-time correctness; everyday training-table work	§1, §2
2. Difficulty profile	pass@1 21.7%, pass@3 25%; consistent, repeatable failures	§4
3. Research awareness	Leakage literature, feature stores, DS-agent benchmarks	§6
4. Scale plan (10 to 1,000)	Parametric generators across mechanism and domain axes, with a headroom screen	§7
5. Failure analysis (bonus)	Per-task diagnosis from the trajectories: genuine difficulty, not task bugs	§5

2 How the tasks are built to resist shortcuts

Every task states the goal and the point-in-time principle, then withholds the method. The agent has to derive the as-of rule itself and apply it consistently across the whole pipeline. That derivation is the difficulty — not data volume, and not any recipe from me. I learned this the hard way: when an instruction named the method outright, the model just applied it, and I had to archive the task (§5, portfolio-twr, is the deliberate version of this).

The shortcut test. A task earned a place in the set only if it can't be solved by one global filter plus a standard `merge_asof` or `groupby`. Concretely, I required three properties: a *per-row cutoff*, since each output row has its own timestamp; *irreducible bitemporality*, two conditions (recorded-before and event-before) that a single-key `merge_asof` can't both express; and *dedup-sensitive aggregation*, a sum or count that comes out wrong if duplicates survive. The test is necessary but not sufficient — a task can pass it and still be easy if the instruction gives the method away, which is exactly what killed two of my candidates.

Why correlated failure is the thing to engineer for. pass@3 passes if *any* of three attempts passes. If failures were independent slips with per-attempt success rate p, pass@3 would be $1-(1-p)^3$, and even $p = 0.11$ would clear the 30% bar. So a low pass@3 only means something if the attempts fail the *same way*. These tasks are built to produce that: next to the correct reading sits a plausible wrong one, and the model takes the wrong one almost every time, so pass@3 stays pinned near pass@1. That's the difference between measuring a capability gap and measuring carelessness.

3 Why the reward is binary

The reward is: does the agent's output pass the full test suite, yes or no. Any failed check nets 0. I want to defend that, because it's the design choice I thought about most.

The test suite has two kinds of checks. There are structural checks — is the output table well-formed, right rows, right columns, sane values — and there are the temporal checks, which grade the wedge itself. The structural bar is easy; any frontier model close to this class clears it nearly every attempt. The temporal checks are the ones I actually care about. So consider what a partial reward would do: I could pad the suite with a hundred wedge checks to drown out the scaffolding, but then the scaffolding checks are just noise in the score, and a fractional reward mostly measures how much padding I chose. Binary is cleaner, and the case analysis is simple:

- If a run fails a **structural** check, either the agent got something fundamentally wrong (a clear fail) or my instruction was under-specified — in which case the fix is on me: refine the instruction or discard the task. I did both during piloting (§7).
- If a run fails a **temporal** check, the agent didn't handle the crux of the problem, which is a definite fail by the definition of the experiment.

The verifier itself is deterministic: it recomputes the answer in a separate container from inputs the agent never sees, and checks each column against a tolerance that forgives rounding but not a point-in-time mistake. I used a deterministic verifier rather than an LLM judge because the answer is exactly computable — a judge would only add noise here. Each task also includes one control column that passes even under a naive approach, which shows the verifier is strict only where the reasoning is.

Task	Domain	reward	pass@1	pass@3	Headroom
credit-feature-pipeline	Consumer credit	0/12	0%	0%	genuine (anchor)
claims-adjudication	Health insurance	0/12	0%	0%	genuine
subscription-churn-labels	B2B SaaS	0/12	0%	0%	genuine
order-ledger-reconstruction	E-commerce	1/12	8.3%	≈25%	genuine (upper anchor)
portfolio-twr	Finance reporting	12/12	100%	100%	none (easy anchor)
Aggregate			**21.7%**	**25.0%**	

Table 2: Per-task and aggregate results, `terminus-2 + gemini-3.5-flash`, 12 trials each. The aggregate pass@3 is the mean of the five per-task values $(0, 0, 0, 0.25, 1.0)$. Harbor reports pass@k at $k \in \{2, 4, 5, 8, 10\}$; the order-ledger pass@3 is interpolated, between pass@2 = 0.167 and pass@4 = 0.333.

4 Difficulty profile

Table 2 has the numbers. Four tasks sit at or near zero and one is fully solved, so the set spans the full range from a hard floor to a working control (Figure 2, left). The right panel is the plot I find most convincing: each dot is one of a task's 12 trials, placed by the share of cells it got wrong in its main failing column, and the dots fall into tight bands. A task's 12 trials nearly all make the same mistake on the same cells — churn is the extreme case, where all 12 trials miss the identical 14 cells. No run crashed or timed out, and the model often spent *longer* on the tasks it failed than on the one it solved, so the difficulty is in the reasoning, not the logistics. The next section reads the trajectories to see what that reasoning actually was.

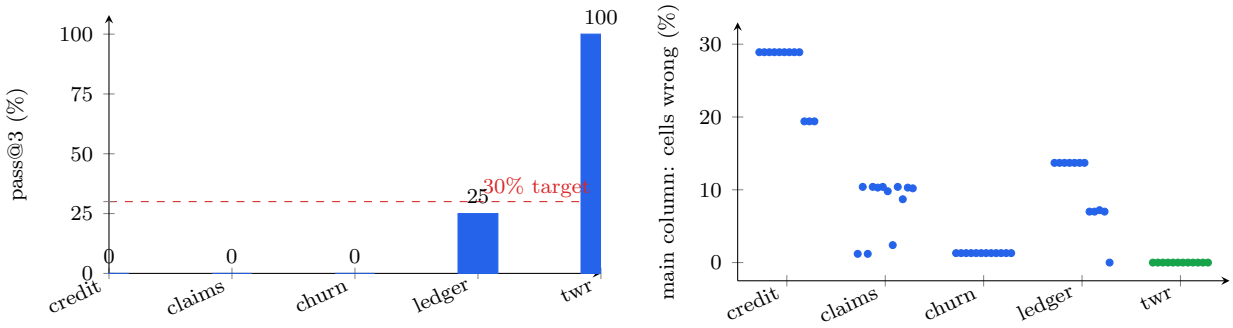


Figure 2: **Left:** pass@3 by task, the difficulty curve, with the 30% target in red. **Right:** one dot per trial, placed by the share of cells wrong in each task's main failing column. The tight bands mean a task's 12 trials repeat the same mistake, which is why pass@3 stays close to pass@1.

5 Failure analysis: they acknowledge it, then don't act on it

Before reading the trajectories I expected the failures to be ignorance — the model not knowing the point-in-time rule. That is not what I found. In task after task the model *states* the rule, sometimes flags exactly the trap I set, and then reasons its way out of applying it. First, the evidence that the mistakes are the model's and not the task's: for every task the reference solution scores 1, the do-nothing agent scores 0, the control column passes, and the truth is recomputed from inputs the agent never sees.

Claims-adjudication. The model got the hard half right and then undid it. It resolved each member's coverage using only records that had reached the payer before the claim was adjudicated — 2390 of 2390 correct on eligibility. At the year-to-date deductible it asked the right question (which prior claims were knowable?), answered it correctly, and then overruled itself: it noted the grader might want the knowability filter and decided against it, on the grounds that *"Option A (adjudicated earlier) is the only option that makes logical business sense."* The code that followed summed every earlier same-year claim with no knowability check, which inflated the deductible and failed about 10% of rows. It saw the trap, named it, and stepped in it anyway.

Subscription-churn. The model computed correctly whether each account was a live subscriber at the observation, and it read the rule that an account which was not live cannot churn and must be left unlabeled. Then it split "label" from "eligibility" and filed the rule on the wrong side: *"This is about the label being 0 ... It doesn't mean they are ineligible."* Its eligibility test reduced to whether the 90-day window had elapsed, and 14 rows came out marked inactive, eligible, and labeled 0 all at once — a combination that cannot be true. All 12 trials produced the same 14 rows, and one trial actually inspected them and judged them correct.

Order-ledger. The one task a single trial solved, and the passing and failing runs differed on exactly one definition. Both handled the two withheld mechanics — counting only events knowable at the checkpoint, and letting a correction void the event it supersedes — and handled them the same way. They split on what *"has been cancelled"* means: the failing run used the order's current status, so an order cancelled and then re-fulfilled read as not cancelled, while the passing run used whether a cancellation had ever happened by the checkpoint. That one choice accounts for all 121 cells where the answer is 1 and the model wrote 0.

Credit-feature-pipeline. "It used event time" would be the lazy description, and it's wrong — the model applied the recorded-time filter to every source. It failed on the snapshot it selected and a step it skipped: it ordered the eligible account snapshots by effective date rather than by what the warehouse would have served, and it never removed duplicate transactions before summing the windows, having checked only the snapshot table for duplicates. Utilization came out above 1.0 on nearly a third of rows. Its self-check was two hand-picked rows, both on the clean path.

Portfolio-twr. The task the model passed every time, and it maps where the difficulty is *not*. It names time-weighted return without giving the method, and my worry was that the model would substitute a more familiar return. It didn't: it derived the right approach from scratch, recognising that the day's cash flow is already inside the closing value and must be removed before measuring growth. So a single well-defined concept, however specialised, is within reach. The headroom in the other four comes from applying a rule consistently across several steps while a plausible wrong reading sits right next to the correct one.

The common thread, and what I think it means. In every failure the model's own checks endorsed the wrong answer: it dismissed a hint aimed straight at the grader's intent (claims), inspected the contradictory rows and approved them (churn), signed off after two clean spot-checks (credit). This is where the lightbulb went off for me. The model clearly *knows about* point-in-time correctness — it recites the rule, because the literature describing the problem (papers, benchmark write-ups, blog posts) is in its training data. What's largely *not* in its training data is worked solutions: those live inside proprietary production systems, or don't get written down at all. So the model has the vocabulary of the problem without the reflexes of the solution, and when the correct

rule conflicts with what "makes logical business sense," the template it actually trained on wins. That's my friend's old heuristic showing up in the trajectories: models are weakest at exactly the things they haven't seen solved. It's also what makes these failures genuine difficulty rather than something a sharper prompt or a retry fixes — the model looks, is satisfied, and fails the same way next time.

6 Research awareness

- **Leakage is the canonical, costly mistake.** Kaufman, Rosset and Perlich (KDD 2011, extended in TKDD 2012) formalise leakage and rank it among the top data mining mistakes; Kapoor and Narayanan (2023) show it is widespread and easy to miss across ML-based science. This is the failure mode the eval grades.
- **Feature stores confirm this is data-science work.** Feast's point-in-time-join documentation exists to ensure future feature values do not leak, and frames the work as freeing data scientists from error-prone joining logic. Tecton, Databricks, and Google's *Rules of Machine Learning* (training/serving skew) make the same point. The pandas `merge_asof` default `allow_exact_matches=True` is the boundary-leak footgun my boundary rows are built to catch.
- **DS-agent benchmarks set the difficulty bar.** DataClawBench — process oriented, native noise preserved, 7 of 8 LLMs under 50% — is the one that shaped my design most: keep the data messy, grade each column separately, treat clean-schema benchmarks as understating difficulty. DataSciBench, InfiAgent-DABench, MLE-bench, DSBench, and DA-Code frame the broader landscape. From SWE-bench Verified I borrowed only the hidden-gold-verifier idea — truth recomputed out of the agent's reach — not the code-editing framing.
- **Takeaway.** Benchmarks that hand the model a clean schema understate difficulty. The signal lives in messy inputs paired with a verifier that recomputes the truth, which is what I built.

7 Scaling from 10 to 1,000 tasks

The tooling already exists. Each task is a seeded generator (`build_inputs.py`) that runs on the host only; the agent never sees it, and the verifier recomputes the truth from a byte-identical copy. A new seed gives a fresh population with the same mechanism, which yields many but closely related variants. Real variety comes from combining a few independent axes.

- **Mechanism**, the axis that carries the difficulty: recorded-time as-of join, slowly-changing-dimension lookup, event-log corrections, right-censored labels, year-to-date accumulators, retroactive corrections. The set covers six; roughly 15 are enumerable.
- **Domain and schema:** credit, claims, churn, commerce, finance, advertising, logistics, grounded in public data (Kaggle credit and churn, LendingClub, CMS claims, the Olist e-commerce set, public event logs) so the realism bar holds.
- **Mess profile and output type:** timestamp encodings, duplicate rate, casing, and whether the output is a feature table, a label table, or a state reconstruction.

Roughly 15 mechanisms across 8 domains, with seeds, comfortably exceeds 1,000 distinct tasks.

The QA loop is the part that actually matters. Quantity is easy to fake and difficulty isn't. Every candidate goes through a three-part screen: the oracle scores 1 and the do-nothing agent scores 0; the task passes the shortcut test from §2; and a frontier model runs 12 trials, keeping only tasks under the pass@3 bar and archiving the ones the model already solves. Failure clustering doubles as an ambiguity check: tightly clustered wrong cells point to a real capability gap, while scattered failures flag a likely ambiguous instruction or verifier bug for human review.

This isn't hypothetical — it's the loop I ran here. I piloted about nine candidates and archived four (clinical-lab, smart-meter, attribution, commission) once the model passed them or an ambiguity surfaced. One of those archived tasks, smart-meter, is the cautionary tale: flash went 0/12 and it looked like headroom, until the trajectories showed the failures came from an ambiguous instruction. Disambiguated, flash passed. Without trajectory reading, that task would have shipped as fake difficulty.

What I automated. Claude Code drove the generator authoring, the host-only input build, the verifier script, the `harbor run` orchestration at 12 trials (alongside the oracle and do-nothing runs), and the parsing of every trajectory into the per-task analyses. What didn't automate was the design: choosing which mechanism to withhold, applying the shortcut test, and reading the trajectories to tell genuine difficulty from a task flaw.

8 Conclusion

The set isolates one skill — point-in-time correctness — and the trajectories show its failures are systematic rather than random: the model states the rule, talks itself out of it, and takes the same plausible wrong turn on nearly every trial. That correlation is what makes 25% pass@3 a real capability signal, and my best explanation for it is a training-data gap: the problem is well described in public text and rarely solved there. Which is also why this slice is worth scaling — the loop in §7 is a repeatable way to find things models can recite but not do, and to manufacture the training signal that closes exactly that gap. Generalising that loop, finding a model's blind spots and generating the data that fixes them, is the part of this problem I find genuinely exciting: it's a concrete step toward systems that improve themselves.